

Contents

SQL Aggregation Prompt Chain	1
Use Case	1
Required Local Preparation	1
Prompt Sequence	2
Related Reference Drawer	2
Relationship To Other SQL Flows	2
SQL Aggregation Chain - 01 Aggregation Intake	2
SQL Aggregation Chain - 02 Output Grain Map	3
SQL Aggregation Chain - 03 Group By And Having Design	4
SQL Aggregation Chain - 04 Aggregation Edge Cases	4
SQL Aggregation Chain - 05 Final Aggregation SQL	5

SQL Aggregation Prompt Chain

Use Case

Use this chain when a human analyst needs Edge Copilot to design Oracle SQL aggregations from explicit business metrics, source fields, grouping grain, filters, **HAVING** rules, null handling, duplicate risk, and validation checks.

The chain is for aggregation design before final SQL. It should define the output grain and metric map before any grouped query is accepted.

Use this chain for:

- COUNT, COUNT DISTINCT, SUM, AVG, MIN, and MAX metrics.
- Conditional aggregation with **CASE** inside aggregate functions.
- Choosing between **WHERE** and **HAVING**.
- Null handling decisions such as keeping nulls, excluding nulls, or using **NVL** only when missing values should behave as zero.
- Avoiding duplicate amplification from joins before aggregation.
- Deciding when analytic/window functions are better than grouped aggregation.

Required Local Preparation

- Gather the business request, metric definitions, acceptance criteria, and sample output if available.
- Prepare relevant table names, source fields, relationship notes, keys, date windows, and known filters.
- Identify candidate output columns and expected reporting grain, such as one row per customer, account, product, day, month, status, or business event.
- Keep row-count notes, duplicate-risk notes, and known null semantics ready.
- Have representative validation checks or reconciliation totals available when possible.

Prompt Sequence

1. `01-aggregation-intake.md`: collect the business request, candidate metrics, source context, grain expectations, and blockers.
2. `02-output-grain-map.md`: require Copilot to define output grain and build the aggregation map before SQL.
3. `03-group-by-and-having-design.md`: design `GROUP BY`, input filters, and `HAVING` rules from the accepted map.
4. `04-aggregation-edge-cases.md`: review null handling, distinct counts, conditional aggregation, join duplicate risk, and analytic/window function alternatives.
5. `05-final-aggregation-sql.md`: produce final Oracle SQL only after grouped result validation checks are defined.

Related Reference Drawer

- `references/oracle-sql/sections/aggregation_patterns.md`

Open this drawer when detailed aggregation guidance is needed. Do not copy long SQL tutorial content from the drawer into Copilot prompts; use it as a local reference for checking the design.

Relationship To Other SQL Flows

- Use `prompt-chains/sql-general/` when the SQL task is broad or not clearly an aggregation-design problem.
- Use this chain after `sql-general` routes a request toward grouped metrics, summary reporting, or aggregation validation.
- Use `prompt-chains/query-tuning/` after the aggregation logic is correct and the main problem is slow runtime, plan evidence, or rewrite performance.

SQL Aggregation Chain - 01 Aggregation Intake

Act as a senior Oracle SQL analyst designing grouped metrics. This is step 1 of a multi-turn aggregation workflow.

This is not yet a final SQL-writing step. First, clarify the business metrics, source context, output grain, and validation needs. Do not write SQL yet.

Business request:

<paste reporting request, metric definitions, stakeholder notes, acceptance criteria, or sample data>

Known source context:

<paste relevant tables, fields, keys, joins, row counts, filters, date windows, and null semantics>

Return:

1. Restated aggregation objective.
2. Candidate business metrics and whether each appears to need `COUNT`, `COUNT DISTINCT`, `SUM`, `AVG`, `MIN`, `MAX`, or conditional aggregation.
3. Candidate output grain, stated as "one row per ...".
4. Candidate source tables and source fields.

5. Known input filters that belong in **WHERE**.
6. Possible group-level rules that may belong in **HAVING**.
7. Null-handling questions for each metric.
8. Duplicate-risk questions, especially where joins may multiply rows before aggregation.
9. Whether grouped aggregation is appropriate, or whether analytic/window functions may be better because detail rows must remain visible.
10. Clarification questions that must be answered before SQL can be written.

Block SQL generation if metric definitions, output grain, source fields, join paths, or validation expectations are incomplete.

SQL Aggregation Chain - 02 Output Grain Map

Continue the Oracle SQL aggregation workflow.

Before writing any SQL, define the output grain and create the aggregation map. The output grain must be explicit and accepted before SQL generation.

Current request and answers:

`<paste the business request, intake answer, and any clarification answers here>`

Schema and relationship notes:

`<paste source tables, fields, keys, join notes, row counts, and null semantics here>`

First return:

1. Proposed output grain, stated as "one row per ...".
2. The exact fields or expressions that define that grain.
3. Any source tables that are at a lower grain than the requested output.
4. Any source tables that could duplicate metric rows if joined before aggregation.
5. Whether any metric should use analytic/window functions instead of grouped aggregation because detail rows need to remain visible.

Then create the aggregation map.

Use this exact aggregation map table shape:

`business metric | source table | source field | filter | grouping grain | aggregate function |`

For each metric, fill one row and explicitly cover:

1. The business definition of the metric.
2. The source table and source field, or * for **COUNT(*)**.
3. Input-row filters that belong in **WHERE**.
4. The grouping grain fields.
5. The aggregate function: **COUNT**, **COUNT DISTINCT**, **SUM**, **AVG**, **MIN**, **MAX**, or conditional aggregation.
6. Any **HAVING** rule that filters aggregate groups after grouping.
7. The output field name.
8. A validation check for the grouped result.

Do not write SQL. If the output grain, aggregate function, null handling, or validation check is missing for any metric, mark the workflow blocked and ask only the questions needed to complete the map.

SQL Aggregation Chain - 03 Group By And Having Design

Continue the Oracle SQL aggregation workflow.

Use the accepted output grain and aggregation map to design the grouped query structure. Do not write final SQL yet.

Accepted output grain:

<paste the accepted "one row per ..." output grain and grain fields here>

Accepted aggregation map:

<paste the completed aggregation map here>

Return:

1. **SELECT** output fields, separating grouped fields from aggregate fields.
2. Required **GROUP BY** fields or expressions. Every non-aggregate selected expression must be accounted for.
3. Input filters that belong in **WHERE** before aggregation.
4. Group filters that belong in **HAVING** after aggregation.
5. Any aggregate expression repeated in **HAVING** and why it belongs there.
6. Conditional aggregation expressions needed for filtered metrics, with the **CASE** logic described in words.
7. Null handling for each metric, including whether nulls should be ignored, counted, preserved, or converted with **NVL**.
8. Any **COUNT(*)**, **COUNT(field)**, or **COUNT DISTINCT field** distinction that affects the business result.
9. Readiness status: ready for edge-case review or blocked.

Rules:

1. Use **WHERE** for input-row filters before aggregation.
2. Use **HAVING** only for rules that depend on aggregate results or grouped rows.
3. Do not use **NVL** to turn nulls into zero unless the business definition says missing values should behave as zero.
4. Do not write final SQL if any grouped field, aggregate expression, or **HAVING** rule is unresolved.

SQL Aggregation Chain - 04 Aggregation Edge Cases

Continue the Oracle SQL aggregation workflow.

Review the aggregation design for correctness risks before final SQL is written.

Current aggregation design:

<paste the output grain, aggregation map, GROUP BY plan, WHERE plan, HAVING plan, and null-handling plan here>

Join and row-grain notes:

<paste join paths, PK/FK notes, source table grains, row counts, or duplicate examples here>

Return an edge-case review with:

1. Output grain confirmation.
2. Null-handling risks for COUNT, SUM, AVG, MIN, and MAX.
3. COUNT(*) versus COUNT(field) versus COUNT DISTINCT field risks.
4. Conditional aggregation risks, including whether false-case values should be 0 or NULL.
5. Duplicate amplification risk from joins before aggregation.
6. Pre-aggregation or deduplication needed before joins.
7. Many-to-many join risks that could inflate metrics.
8. HAVING versus WHERE mistakes to correct.
9. Cases where analytic/window functions are better than grouped aggregation because detail rows should remain visible.
10. Validation checks that must be run before accepting the final SQL.

For validation checks, require checks for:

1. Output grain uniqueness.
2. Reconciliation between detail totals and grouped totals where applicable.
3. Null impact, such as non-null counts versus row counts.
4. Distinct-count key validity.
5. Join amplification, such as row counts before and after joins.

Do not write final SQL if any edge-case risk changes the metric definition, grouping grain, join order, or validation plan.

SQL Aggregation Chain - 05 Final Aggregation SQL

Continue the Oracle SQL aggregation workflow.

Write final Oracle SQL only if the output grain, aggregation map, GROUP BY fields, WHERE filters, HAVING rules, null handling, duplicate-risk handling, and validation checks have all been defined.

Approved aggregation design:

<paste the accepted output grain, completed aggregation map, GROUP BY plan, HAVING plan, edge-c

Return:

1. Readiness status: ready or blocked.
2. If blocked, list only the missing decisions and do not write SQL.
3. If ready, write the final Oracle SQL.
4. After the SQL, list each metric and how it maps back to the approved aggregation map.
5. List the grouped-result validation checks that must pass before the SQL is accepted.
6. List any assumptions that remain and the risk of each assumption.

Final SQL requirements:

1. Preserve the approved output grain.
2. Include only grouped fields and aggregate expressions in the grouped SELECT.
3. Put input-row filters in WHERE.

4. Put aggregate group filters in **HAVING**.
5. Handle nulls exactly as approved.
6. Prevent duplicate amplification using the approved pre-aggregation, deduplication, or join sequence.
7. Use analytic/window functions instead of grouped aggregation only when the approved design says detail rows must remain visible.
8. Use clear aliases for output fields.

Do not accept the final SQL unless validation checks for grouped results are included with it.